

level 1說明

背景介紹：

我們實作的邏輯是利用 python 的 pulp 函式庫，他是一個線性求解器，假設我們對他輸出輸入多維度的線性限制式與目標函數，他就可以幫我們解求符合目標函數的最佳解。在這套系統中，我們將 Constraint 輸入進求解器，並將求解目標設為是最小化 ($10000 \times \text{aperiodic task miss} + (-\text{售電金額}) + \text{發電成本}$)，讓求解器求出這72個小時中，每個小時的任務、發電等情況的全局結果為何。

Acceptance Test邏輯(4-1)：

基礎排程與資源鎖定：針對具備最高可預測性的 Periodic tasks，系統於日前排程階段 (Base Schedule) 會優先為其規劃完整的 72 小時執行與供電方案，尋求發電成本之最小化。排程完成後，系統會將其執行時段與電力來源 (指定發電設備) 強制固定，確保 Periodic tasks 絕對不會因為新任務加入而做不完。此設計完美符合 Assumption 6 (Sporadic task 只能在不取代、不移動、不重新最佳化既有 Periodic jobs 的前提下插入) 的規定。

動態邊界與模型重建：既然 Periodic tasks 的資源已硬性鎖定，當 Sporadic 或 Aperiodic tasks 抵達觸發 Acceptance Test 時，求解引擎只能在「鎖定後剩餘的可用機組出力餘裕」與「剩餘儲能空間」這兩項資源中進行重新最佳化。系統會為新任務生成對應的執行變數 (x) 與供電來源變數 (k)，拆除舊的全局能量平衡限制式，並將新需求納入全新的 Global Balance 中。

決策機制與虛擬懲罰：系統為 Sporadic 任務綁定了一個二元決策變數 Reject_j ，並在目標函數中賦予極高的虛擬懲罰權重 ($1,000,000$)。雖然作業說明中拒絕 Sporadic task 並無實質懲處，但此高額設定更符合現實世界中緊急任務的絕對重要性，同時也是因為 sporadic acceptance rate 會影響到這份專案的分數。

最終判斷標準：當求解器重新計算 r 到 72 小時的排程時，會盡全力調整發電機的 Ramp-up 或釋放儲能電量來滿足新任務，以避免百萬懲罰。若求解器能找到一組解使 $\text{Reject}_j = 0$ ，系統即接受 (Accept) 該任務；反之，若因物理限制絕對無法排入，迫使 $\text{Reject}_j = 1$ ，則判定拒絕 (Reject)。

日前保留策略與實際效能數據驗證(6-1)

基於成本最佳化之天然餘裕保留策略：本系統並未採用盲目預留特定發電容量的靜態作法，而是透過日前排程階段追求「發電成本最小化」來實現天然的動態保留。為了壓低成本，系統會避免不必要的機組滿載運轉，並在電價或成本較低時將電能存入儲能設備。這種不輕易滿載的經濟調度，自然而然為系統保留了極大的 Ramp-up 爬升空間與電池 SOC 餘裕。

實際效能數據驗證：從排程結果的實際數據來看，此策略發揮了卓越的防禦效果。面對測試資料中極度嚴苛的 Sporadic tasks（例如 s1 至 s5 皆要求連續 3 小時、每小時高達 20 MWh 的龐大電量，且設定為 preempt=0 不可中斷），系統憑藉日前排程留下的動態餘裕，成功達成了 100% 的接納率，且 Sporadic Value Rate 高達 0.9333（有加入一個 $w = 200$ 的 boss sporadic task）。這證明此「基於確定性鎖定與成本最佳化」的保留策略，能完美應付高強度的突發用電需求。

目標函數排擠效應與權衡分析(6-2)

在本模型的目標函數中，提升 Sporadic 與 Aperiodic tasks 的接納率，必然會對發電成本與售電收益產生強烈的排擠效應與權衡拉扯。

1. 極大化接納率 vs. 發電成本增加：為確保所有緊急任務順利執行（規避 10,000 元的 Miss 懲罰與百萬元的 Reject 懲罰），系統在 Acceptance Test 期間必須動用剩餘的機組 Ramp-up 能力，甚至被迫啟動成本較為高昂的傳統備援機組，這直接導致了整體發電成本的上升。
2. 極大化接納率 vs. 售電收益下降：當突發任務抵達時，系統會將儲能設備的電量優先供給這些任務。這意味著，原本可以在高電價時段「售出至市場進行套利」的寶貴電能被內部消耗掉了，進而壓縮了整體的市場售電收益。
3. 實際數據權衡結論：根據 72 小時的最終排程結果，系統為了維持 0.0 的 Hard/Soft deadline miss rate，在遭遇突發任務時果斷犧牲了部分的市場套利空間與最低成本配置。然而，由於 MILP 求解器強大的全局調度能力，最終系統仍繳出了發電成本 307,209.6 元、市場售電收益 380,267.0 元的成績。這顯示系統在「極端防禦（100% 接收率）」的狀態下，依然透過精準的充放電與發電調度，達成了 -73,057.4 的總目標函數值。虛擬電廠不僅完美吃下所有負載，實質上還淨賺了 7 萬餘元，完美實現了三大目標函數之間的最佳平衡。

scheduler.py

1. 將各種 json 檔載入

a. load_task()

讀取"output/task_set.json", 這個檔案

b. load_un_task()

讀取"input/aperiodic_n_sporadic.json", 這個檔案

c. load_environment()

讀取"input/processor_settings.json", 這個檔案

2. Constraint 建立

a. _build_generator_constraints() 6~12

- i. [Constraint 8]: 最小出力 $\leq$ ramp up
- ii. [Constraint 11]: 排程前的關機時間限制
- iii. [Constraint 12]: 排程前的開機時間限制
- iv. [Constraint 9]: 最小開機時數限制
- v. [Constraint 10]: 最小關機時數限制
- vi. [Constraint 6]: 最大/最小出力的上下限
- vii. [Constraint 7]: ramp_up / ramp_down 的限制

b. _build_storage_constraints() 13~18

- i. [Constraint 15]: 最大充電限制
- ii. [Constraint 14]: 最大放電限制
- iii. [Constraint 17]: 儲能設備的儲能 上下限
- iv. [Constraint 18]: 不能放出超過最低存量的電能
- v. [Constraint 16]: 電量守恒限制
- vi. [Constraint 13]: 再生能源預測電量限制

c. _build_job_constraints() 1~5

- i. aperiodic

他有一個自己的 miss屬性、Drop屬性。

miss。

如果這個任務在72小時的排程內完全做不完 $\text{drop} = 1$

如果這個任務在他的deadline內做不完 $\text{miss} = 1$

如果 $\text{miss} = 0$ ，那他要在他的 $r \sim d$ 之間做完 e 個時間

1. [Constraint 4]: miss 的限制, $\text{miss} = 0$ 期限內要做完, $\text{miss} = 1$ 72小時結束前要做完

2. [Constraint 2]: release time 的限制

ii. sporadic

他有自己的 **reject**屬性。

$\text{reject} = 0$ ，則他需要在 $r \sim d$ 之間做完 e 個時間

1. [Constraint 2]: release time 的限制

2. [Constraint 3]: deadline 的限制

iii. Periodic

1. [Constraint 2]: release time 的限制

2. [Constraint 3]: deadline 的限制

iv. 共通

1. [Constraint 5]: non-preemptive 要連續執行，他的邏輯是追蹤這個task，在整個時間週期間，執行變成不執行 跟 不執行變成執行 的次數，如果是preemptive 這個次數最多只能 2 次（開始執行 + 結束執行）

2. [Constraint 1]: 有執行就會要給他足夠的電

d. 其他constraint 19~23

i. [Constraint 19]: 宣告階段即將電池充電的變數設為 binary變數

ii. [Constraint 20]: `_apply_dynamic_balance()`

iii. [Constraint 21]: `_apply_dynamic_balance()`

iv. [Constraint 22]: 宣告階段即將 Sell 變數的 low bound設為 0，避免了售電量為負值的情況。

v. [Constraint 23]: `_apply_dynamic_balance()`

3. 週期性任務的排程

4. `run_base_schedule()`

先去執行 `self._apply_dynamic_balance()` 算出 periodic tasks 的排程，之後再去執行

`self.lock_scheduled_jobs( )` 鎖定 periodic task 的排成

5. `_apply_dynamic_balance( )`

a. 限制式建立

- i. `GenLimit_{i}_{t}`: 所有任務在 t 時間點從 i 發電機拿的電 $\leq i$ 發電機實際生產的值
- ii. `ResLimit_{i}_{t}`: 所有任務在 t 時間點從 i 再生能源拿的電 $\leq i$ 再生能源實際生產的值
- iii. `StoLimit_{sid}_{t}`: 所有任務在 t 時間點從 i 電池拿的電 $\leq i$ 電池實際給的值
- iv. `NoBat2Bat_{t}`: 所有電池在 t 時間點拿的電能 $\leq$ 再生能源 + 發電機 - task 用電
- v. `GlobalBal_{t}`: 在時間 t 時, 所有供電 = 所有耗電

b. 只要有新的耗電任務進來就會把原本的限制式刪除, 重新計算溪的平衡。

6. `lock_scheduled_jobs( )`

鎖定已經排程好的 Periodic tasks, 他排在哪個時間做就一定要做, 不能改變

7. 非週期性任務的排程

會先在 main function 中執行

`scheduler.process_unexpected_jobs(unexpected_set)` 處理動態任務。

a. `process_unexpected_jobs( )`

- i. 先按照 release time 先後順序 sort, 接下來要一個一個遍歷
- ii. lock 前面的排程 (因為排程器不應該知道未來會突然有非週期任務, 故前面的排序要維持不變)
- iii. 將這個變數的基本參數加到 var 中
- iv. 透過 `self._build_job_constraints(job_dict) + self._apply_dynamic_balance() + self._update_objective()` 這三個function可以建立系統新的全局平衡

- v. `self.model.solve(pulp.PULP_CBC_CMD(msg=False))`
透過這一行進行求解（運算出一個加入新的 task 後的最佳解）
- vi. **sporadic (hard deadline)**
 - 1. `rejected`: 將他加到 `rejected_sporadic` 的 list 中
 - 2. `not rejected`: 整理哪幾個時間會被執行，然後輸出到 `acceptance log` 裡
- vii. **aperiodic (soft deadline)**

- b. `_lock_past_states( )`
 - i. for loop 由 上一次的 lock time + 1 ~ current_t
(該非週期任務的 release time)
 - ii. 發電機在 t 的發電是多少就是多少，不能改變。
 - iii. 鎖住電池在 時間 t 的電存量 + 充電量 + 放電量

level 2改動

既然原本將 periodic 鎖定可以求出解，那在level 2中解除他們的鎖定只是讓解的數量變多，原本的解必定存在於更多的解的集合中，所以解除鎖定必也能夠找到解

紀錄

第一次改動：

單純註解掉 `lock_scheduled_jobs()` 讓排程器可以重新最佳化 periodic tasks。

問題：因為解除 periodic tasks 的鎖定，整個 model 要被決定的變數過多，導致排程器遇到一個非週期性任務就想很久（約30秒/個）。

解法：設置排程器只能想 8 秒 (`TimeLimit = 8`)。

結果：算出來的結果反而比固定 periodic task 的結果差（數值越低越好）。

- 原結果：\$ -47019.21
- 改動後：\$ -46407.77

第二次改動：

加入 Frame 的概念，假設一個突發任務在 $t=5$ 出現 deadline 為 $t=8$ ，那只需要解開

$t=[5, 8]$ 的限制式，或是找到

假設這個非週期的 task 可以執行的區間為 $t=[5, 8]$ ，那我們就解開 $\text{release time} \sim \text{abs_deadline}$ 有可能在這個區間的 task，然後加上離開的限制式， $\text{SOC} \geq$ 原本的 SOC。那這邊發電機的出力也要 $\geq$ 原本的嗎，如果要的話，可以幫我補上所有會需要加上限制式的變數嗎

需要注意，因為未來的限制式是固定的，所以如果解開 $t=[5, 8]$ ，那要確保

1. $\text{SOC}[t=12] \geq$ 剛剛重新最佳化的 $\text{SOC}[t=12]$
2. 剛剛重新最佳化的 $\text{SOC}[t=12] +$ 他的 ramp_up/down 的能力要可以 $= P[i, t]$
3. 開機狀態
 - a. 第一步先看鎖定開關機狀態能不能有解
 - b. 不能就解開未來的所有變數

問題：算出來的結果跟原本比差得很多，可能是因為只解開該 frame 可以產生出局部最佳解，但是這個局部解無法導向全局最佳解（因為 miss 了很多個 aperiodic tasks）。

結果：

- 原結果：\$ -47019.21
- 改動後：\$ -12648.72

第三次改動：

跟第一版相同，只是將 timelimit 設為 10，得到的結果與第一次毫無差異。

將 timelimit 設為 15，可以得到 \$ -47566.96 的結果

可以得出目前如果不讓排程器想超過 10 秒得出的結果，不會比鎖定 periodic tasks 的結果更好

應該是要朝向減少不確定性的方向來解

目前的最佳解：

不設 timelimit 的情況可以得到數據為 \$ -48200.83，是理論上無法預知非週期性任務下的最佳解，但並不是全局最佳解，全局最佳解是預先知道所有非週期性任務的上帝視角最佳解

放寬的Assumptions :

1. sporadic jobs 加入後會在確保不miss原本periodic tasks的情況下, 重新最佳化並移動periodic tasks達到更好的排程效果 (完成)

2. 再生能源實際生產電力的結果會落在預測值的 $\pm 5\%$ 以內 (完成)
將限制式13修改如下

$$0 \leq P_{i,t} \leq \text{renewmax}_i * \Delta t * \text{renewforecast}_{i,t} * \text{floatingcons}_{i,t} \quad \forall i \in Ir, \forall t \in T, \forall \text{floatingcons} \in [0.95, 1.05]$$

3. 售電價格會落在預測值的 $\pm 5\%$ 以內 (完成) (generator 修改)
新增限制式

$$\lambda_t^{\text{float}} = \lambda_t * \text{pricefloatingcons}_t \quad \forall \lambda_t \in \mathbb{Z}^+ \cup 0, \forall \text{pricefloatingcons} \in [0.95, 1.05]$$

4. 一開始排程會先簽訂售電契約 (單位為每個小時) (不知道這能不能算兩個) (完成) (generator 修改)

a. 售電量 > 簽訂值 : 多出來的電以當下的電價 0.7 倍售出

b. 售電量 < 簽訂值 : 少一度電罰款 100 元

跟熱門時段一起算

$$f_3 = - \left(\sum_{t \in T \setminus \text{Peak}} \begin{cases} \lambda_t^{\text{float}} * \text{Sell}_t^{\text{plan}} + 0.7 * \lambda_t^{\text{float}} * (\text{Sell}_t^{\text{actual}} - \text{Sell}_t^{\text{plan}}) & \text{if } \text{Sell}_t^{\text{actual}} \geq \text{Sell}_t^{\text{plan}} \\ \lambda_t^{\text{float}} * \text{Sell}_t^{\text{actual}} - 100 * (\text{Sell}_t^{\text{plan}} - \text{Sell}_t^{\text{actual}}) & \text{if } \text{Sell}_t^{\text{actual}} < \text{Sell}_t^{\text{plan}} \end{cases} \right) + \sum_{t \in \text{Peak}} \begin{cases} \lambda_t^{\text{float}} * \gamma * \text{Sell}_t^{\text{plan}} + 0.7 * \gamma * \lambda_t^{\text{float}} * (\text{Sell}_t^{\text{actual}} - \text{Sell}_t^{\text{plan}}) & \text{if } \text{Sell}_t^{\text{actual}} \geq \text{Sell}_t^{\text{plan}} \\ \lambda_t^{\text{float}} * \gamma * \text{Sell}_t^{\text{actual}} - 100 * (\text{Sell}_t^{\text{plan}} - \text{Sell}_t^{\text{actual}}) & \text{if } \text{Sell}_t^{\text{actual}} < \text{Sell}_t^{\text{plan}} \end{cases}$$

$$\text{Peak} \subset T, \gamma = 1.25$$

, $\text{Sell}_t^{\text{plan}}$ 為排程時預先簽訂該時段要賣的電量

, $\text{Sell}_t^{\text{actual}}$ 為該時段實際賣的電量

5. period task之間有先後順序關係的限制 (因為每個 period task 之間的週期可能都不相同, 所以有點難設定那些任務要先執行, 可能要去修改 task_generator 或是換一個)
6. 電池放電效率會受到放電輸出大小影響, 在輸出功率過最大輸出入功率80%以上便會受到影響(公式: (這個限制好像會讓求解器求解的時間複雜度大幅上升, 可能會花太多時間))

$$\text{實際消耗} = \max \left(\text{預定輸出}, \left(1 + \frac{\text{預定輸出} - \text{最大功率} * 0.8}{\text{最大功率}} \right) * \text{預定輸出} \right)$$

7. 電池充電效率會受到充電輸出大小影響, 在輸入功率過最大輸出入功率80%以上便會受到影響(公式: (這個限制好像會讓求解器求解的時間複雜度大幅上升, 可能會花太多時間))

$$\text{實際充入} = \min \left(\text{預計充入}, 1 - \frac{(\text{預計充入} - \text{最大功率} * 0.8)}{\text{最大功率}} * \text{預計充入} \right)$$

8. 電池自放電, 電池每小時會流失一定比例的電量(限制式最後是對於所有t屬於T) (完成)我這邊寫每個時間單位會漏電1%

$$SOC_{i,t} = SOC_{i,t-1} \cdot (1 - \sigma_i) + \sum_{j \in J_{chg}: b(j)=i} \sum_{l \in I_g \cup I_r} k_{j,l,t} - P_{i,t} \quad \forall i \in I_b, \forall t \in T$$

9. 電池老化成本:每次充放1MWh的電需花費一定成本(完成)我這邊的成本設為5(generator 修改)

$$\begin{aligned} \text{目標改為 } \min F &= \alpha f_1 + f_2 + f_3 + f_{age} \\ f_{age} &= \sum_{t \in T} \sum_{i \in I_b} cost_{age} * (P_{i,t} + \sum_{j \in J_{chg}: b(j)=i} \sum_{l \in I_g \cup I_r} k_{j,l,t}) \\ cost_{age} &= 5, \text{ 定義為儲能設備每充放電 1MWh 的老化成本} \end{aligned}$$

10. 電池保護機制:當電池以存容量超過其最大容量的80%時, 其最大輸入功率會依比例下降(完成)

$$\text{目前功率} = \min \left(\text{最大功率}, 1 - \frac{\text{目前電量} - \text{最大電量} * 0.8}{\text{最大電量} * 0.2} * \text{最大功率} \right)$$

$$\sum_{j \in J_{chg}: b(j)=i} \sum_{l \in I_g \cup I_r} K_{j,l,t} \leq chg_i * \Delta t * \left(1 - \frac{SOC_{i,t-1} - stomax_i * 0.8}{stomax_i * 0.2} \right) \quad \forall i \in I_b, \forall t \in T$$

11. 電池保護機制:當電池以存容量低於其最大容量的20%時, 其最大輸出功率會依比例下降 (完成)

$$\text{目前功率} = \min \left(\text{最大功率}, \left(1 + \frac{\text{目前電量} - \text{最大電量} * 0.2}{\text{最大電量} * 0.2} \right) * \text{最大功率} \right)$$

$$P_{i,t} \leq dis_i * \Delta t * \left(1 + \frac{SOC_{i,t-1} - stomax_i * 0.2}{stomax_i * 0.2} \right) \quad \forall i \in I_b, \forall t \in T$$

12. 熱門時段售電獎勵:以random在price_72hr.json隨機(也許每24小時選取3小時)在售電價不為零的時段定義新變數peak決定是否為突發用電高峰, 若在此時段售電則可以賺取原價的1.25倍電價 (完成)

13. 加班費懲罰:定義深夜時段(也許12-18, 36-42, 60-66), 在此時段若傳統發電廠是啟動狀態, 則固定成本為原固定成本的1.25倍 (完成)

(這個1.25倍 我設定成只對開機的成本"cost_fixed" * 1.25倍, 生產多少電"cost_variable" 不受這個條件影響)
(時間: [12, 19] + [36, 43] + [60, 67])

$$f_2 = \sum_{i \in I_g} \left(\sum_{t \in T} costup_i * P_{i,t} + \sum_{t \in T \setminus OT} coston_i * U_{i,t} + \sum_{t \in OT} coston_i * U_{i,t} * \rho \right)$$

加班時間集合 $OT \subset T (t \in [12, 19] \cup [36, 43] \cup [60, 67])$
 $\rho = 1.25$ 加班費係數

- 14.

待整理 & 新增內容：

我們在leve2針對了幾個部份做了優化，並且努力找出其中可以達到最佳排程的方案

1.直接解放 **Periodic tasks**

當非週期任務來時，重新最佳化 $t = \text{release time} \sim$ 結尾的所有tasks的排程結果。

優點：這個做法理論上可以得到不知道非週期任務什麼時候會來條件下的最佳解。

缺點：因為解放periodic task，需要花費的計算時間會較長

2.解放部份 **periodic tasks**

假設非週期任務從 r 開始 d 為deadline，那我們重新最佳化的區間就取從 $r \sim \max(\text{所有可能在這個時間區段被執行任務的deadline})$ 。

優點：相對於第一個排程速度提高很多

缺點：成果下降非常多，可以想成是因為只看小區間只能得到局部最佳解，但是無法達到全局的最佳解

3.全部解放但限制時間（15秒）

預估發電總成本：\$ 308775.93

預估售電總收益：\$ 356342.89

系統真實淨成本（不含虛擬罰款）：\$ -47566.96

目前這個應該是考慮效能以及時間下的較佳方案，但同時也是因為現在限制時間只有72 小時，如果時間多timelimit設為15秒，不一定可以取得比“不解放periodic tasks”更好的結果

4.全部解放但限制時間 + 精度放寬（15秒 & 1%）

預估發電總成本：\$ 322921.42

預估售電總收益（排程內）：\$ 362019.16

系統真實淨成本（含 Bonus）：\$ -37813.86

好像是可以稍微快一點，輸出的成果跟原本差不多

[assumption 1]

加入+-5%的再生能源限制結果（不須修改evaluator即可完成）[assumption 2]

預估發電總成本：\$ 319590.08

預估售電總收益（排程內）：\$ 363031.46

真實天氣溢出收益 (Bonus): \$ 3691.16
系統真實淨成本 (含 Bonus): \$ -47132.53
Rejected Sporadic 數量: 0
Missed Aperiodic 數量: 0

加入電池自放電的結果 (每個時間單位漏電1%) [assumption 8]
最終求解狀態: Optimal
預估發電總成本: \$ 319796.33
預估售電總收益 (排程內): \$ 359498.59
真實天氣溢出收益 (Bonus): \$ 3440.15
系統真實淨成本 (含 Bonus): \$ -43142.41
Rejected Sporadic 數量: 0
Missed Aperiodic 數量: 0

加入電池老化成本 & 電池保護機制的結果[assumption 9]+[assumption 10]
最終求解狀態: Optimal
預估發電總成本: \$ 322886.38
電池老化總成本: \$ 4625.13
預估售電總收益 (排程內): \$ 362009.89
真實天氣溢出收益 (Bonus): \$ 3056.25
系統真實淨成本 (含 Bonus): \$ -37554.63
Rejected Sporadic 數量: 0
Missed Aperiodic 數量: 0

加入售電價格 +-5% & 售電契約機制 [assumption 3]+[assumption 4]
最終求解狀態: Optimal
預估發電總成本: \$ 344282.18
電池老化總成本: \$ 4609.33
預估售電總收益 (排程內): \$ 374898.20
實際售電總收益 (含 +-5% 波動與 0.7 倍折價): \$ 371636.36
售電違約總罰金: \$ 9764.91
真實天氣溢出收益 (Bonus): \$ 2971.06
系統真實淨成本 (含 Bonus): \$ -15951.00
Rejected Sporadic 數量: 0
Missed Aperiodic 數量: 0

加入電池放電保護機制 [assumption 11]

最終求解狀態: Optimal

預估發電總成本: \$ 344309.12

電池老化總成本: \$ 4647.70

預估售電總收益 (排程內): \$ 376881.60

實際售電總收益 (含 $\pm 5\%$ 波動與 0.7 倍折價): \$ 379287.67

售電違約總罰金: \$ 9373.04

真實天氣溢出收益 (Bonus): \$ 2711.65

系統真實淨成本 (含 Bonus): \$ -23669.46

Rejected Sporadic 數量: 0

Missed Aperiodic 數量: 0

加入加班費機制 [assumption 13]

最終求解狀態: Optimal

預估發電總成本: \$ 381197.74

電池老化總成本: \$ 5103.99

預估售電總收益 (排程內): \$ 419892.27

實際售電總收益 (含 $\pm 5\%$ 波動與 0.7 倍折價): \$ 418782.71

售電違約總罰金: \$ 15048.86

真實天氣溢出收益 (Bonus): \$ 3410.18

系統真實淨成本 (含 Bonus): \$ -20842.31

Rejected Sporadic 數量: 0

Missed Aperiodic 數量: 0